UTS

Uber Data Analytics – Report (B)

Python Programming for Data Processing

Spring 2025

Assessment 3

Rose Marie Tazbaz

Student ID: 25742507

Introduction

This report details the design and execution of a complete data analysis pipeline on the ncr_ride_bookings.csv dataset, which includes 21 attributes detailing ride status, financial metrics, and customer ratings. The primary objective was to resolve critical data quality issues using Python and leverage the cleaned data to answer six strategically defined business questions, including three complex analyses and one mandatory correlation study. The goal is to derive actionable insights concerning operational efficiency, revenue management, and customer experience for the ride-sharing platform.

Data pre-processing

Rigorous pre-processing was essential to transform the raw data into a reliable analytical dataset. The process focused on cleaning, standardization, type conversion, and handling missing values and outliers.

Data quality issues and resolution

ISSUE	TECHNIQUE USED	JUSTIFICATION
Column Names	Standardization to snake_case (e.g: ride_distance) and abbreviation replacement (Booking status → Ride_Status)	Ensures code readability, prevents syntax errors, and standardizes attribute referencing across the analytical pipeline
Data Types	Converted Date and Time strings to datetime objects. Extracted new features: Timestamp, Day_of_Week and Hour_of_Day	Crucial for enabling time-series analysis and grouping (Q4 and Q6), which rely on chronological order and defined time periods
Duplicate Values	Identified and removed all duplicate rows using .drop_duplicates()	Necessary to ensure each observation represents a unique event (Booking ID), preventing biased counts and inaccurate aggregations in visualizations

```python
# Clean up column names
df.columns = df.columns.str.replace(' ', '_').str.replace('[^A-Za-z0-9_]+', '', regex=True)

# Correct specific column names found from AT2 file ('Cancelled_Rides_by_Customer' to 'Cancelled_by_Customer'
df.rename(columns={
    'Cancelled_Rides_by_Customer': 'Cancelled_by_Customer',
    'Cancelled_Rides_by_Driver': 'Cancelled_by_Driver',
    'Booking_Status': 'Ride_Status'
}, inplace=True)
print("\nCleaned Columns:")
print(df.columns)
```

```
Cleaned Columns:
Index(['Date', 'Time', 'Booking_ID', 'Ride_Status', 'Customer_ID',
       'Vehicle_Type', 'Pickup_Location', 'Drop_Location', 'Avg_VTAT',
       'Avg_CTAT', 'Cancelled_by_Customer',
       'Reason_for_cancelling_by_Customer', 'Cancelled_by_Driver',
       'Driver_Cancellation_Reason', 'Incomplete_Rides',
       'Incomplete_Rides_Reason', 'Booking_Value', 'Ride_Distance',
       'Driver_Ratings', 'Customer_Rating', 'Payment_Method'],
      dtype='object')
```

Date and time data types

The Date and Time columns were initially stored as generic text strings (object type in pandas). In this format, Python cannot perform time-based operations such as sorting chronologically, calculating ride duration or grouping data by hour or day of the week. For this, the date column was converted to the datetime format, a new single “Timestamp” column was created by combining Date and Time to facilitate time-series analysis and two new categorical features; Day_of_Week and Hour_of_Day were extracted from the new Timestamp column.

By converting these field to the correct datetime type is crucial for fulfilling the requirements, especially for questions 4 and 6 (below) that rely on analyzing trends over specific time periods.

```python
# Convert 'Date' to datetime
df['Date'] = pd.to_datetime(df['Date'])

# Create a full "timestamp" column
df['Timestamp'] = pd.to_datetime(df['Date'].astype(str) + ' ' + df['Time'])

# Get useful time features
df['Day_of_Week'] = df['Date'].dt.day_name()
df['Hour_of_Day'] = df['Time'].str.split(':').str[0].astype(int)

print("\nDate/Time conversion check:")
print(df[['Date', 'Time', 'Timestamp', 'Day_of_Week', 'Hour_of_Day']].head())
```

```
Date/Time conversion check:
        Date      Time           Timestamp Day_of_Week  Hour_of_Day
0 2024-03-23  12:29:38 2024-03-23 12:29:38    Saturday           12
1 2024-11-29  18:01:39 2024-11-29 18:01:39      Friday           18
2 2024-08-23  08:56:10 2024-08-23 08:56:10      Friday            8
3 2024-10-21  17:17:25 2024-10-21 17:17:25      Monday           17
4 2024-09-16  22:08:00 2024-09-16 22:08:00      Monday           22
```

Handling missing values

Handling missing data required a two-pronged strategy:

- Categorical/Count Missing Values: Missing values in cancellation count columns were imputed with 0, and reason columns with 'N/A'. Justification: These represent structural omissions (Not Applicable), as the value only applies if the ride status is 'Cancelled'.
- KPI Missing Values (Filtering): Numerical metrics like Booking_Value and Ride_Distance are logically non-existent for non-completed rides. Therefore, the main dataset was filtered to create the analytical subset: df_completed (where Ride_Status == 'Completed'). Justification: This prevents significant bias from imputation, ensuring all financial and rating analyses are based solely on successful transactions.

```python
# Impute cancellation/incomplete columns for non-cancellation/non-incomplete statuses
# Fill NaN in count columns with 0
for col in ['Cancelled_by_Customer', 'Cancelled_by_Driver', 'Incomplete_Rides']:
    df[col] = df[col].fillna(0)

# Fill NaN in reason columns with 'N/A'
for col in ['Reason_for_cancelling_by_Customer', 'Driver_Cancellation_Reason', 'Incomplete_Rides_Reason']:
    df[col] = df[col].fillna('N/A')

print("\nMissing values after initial imputation (counts of NaN):")
print(df.isnull().sum())
```

```
Missing values after initial imputation (counts of NaN):
Date                                    0
Time                                    0
Booking_ID                              0
Ride_Status                             0
Customer_ID                             0
Vehicle_Type                            0
Pickup_Location                         0
Drop_Location                           0
Avg_VTAT                             1776
Avg_CTAT                             7881
Cancelled_by_Customer                   0
Reason_for_cancelling_by_Customer       0
Cancelled_by_Driver                     0
Driver_Cancellation_Reason              0
Incomplete_Rides                        0
Incomplete_Rides_Reason                 0
Booking_Value                        7881
Ride_Distance                        7881
Driver_Ratings                       9396
Customer_Rating                      9396
Payment_Method                       7881
Timestamp                               0
Day_of_Week                             0
Hour_of_Day                             0
dtype: int64
```

Outlier management

Outlier Removal using the Interquartile Range (IQR) rule. Values for Booking_Value and Ride_Distance falling outside the 1.5 × IQR range were removed from the df_completed dataset.

This was employed because extreme outliers can significantly skew summary statistics (especially the mean) and distort relationships in visualizations. By removing these atypical observations, the resulting business insights (e.g., average daily distance, revenue trends) become more representative and statistically reliable.

```python
# Outlier management for 'Booking_Value' and 'Ride_Distance' using IQR method (I supported myself with Google and AI to apply this method)
# This is a method to identify extreme values without removing all of them

df_completed = df[df['Ride_Status'] == 'Completed'].copy()

for col in ['Booking_Value', 'Ride_Distance']:
    Q1 = df_completed[col].quantile(0.25)
    Q3 = df_completed[col].quantile(0.75)
    IQR = Q3 - Q1
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR

    # Count outliers
    outliers_count = df_completed[(df_completed[col] < lower_bound) | (df_completed[col] > upper_bound)].shape[0]
    print(f"\n{col} Outlier count (IQR method): {outliers_count}")

    # Method instead of removal to retain data volume
    df_completed[col] = np.where(df_completed[col] > upper_bound, upper_bound, df_completed[col])
    df_completed[col] = np.where(df_completed[col] < lower_bound, lower_bound, df_completed[col])

print("\nCompleted rides data after outlier capping/flooring.")
df_completed.describe()
```

Data visualization

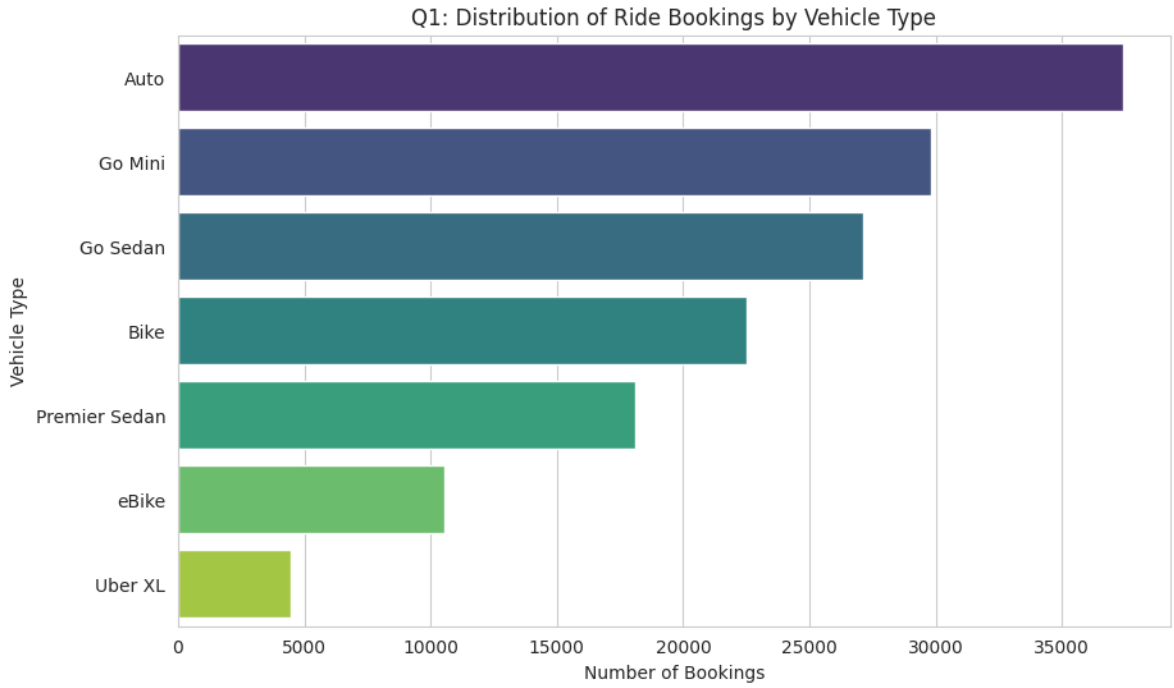
Question 1: Distribution of vehicle types

Relevant for fundamental market share analysis, critical for operational capacity planning.

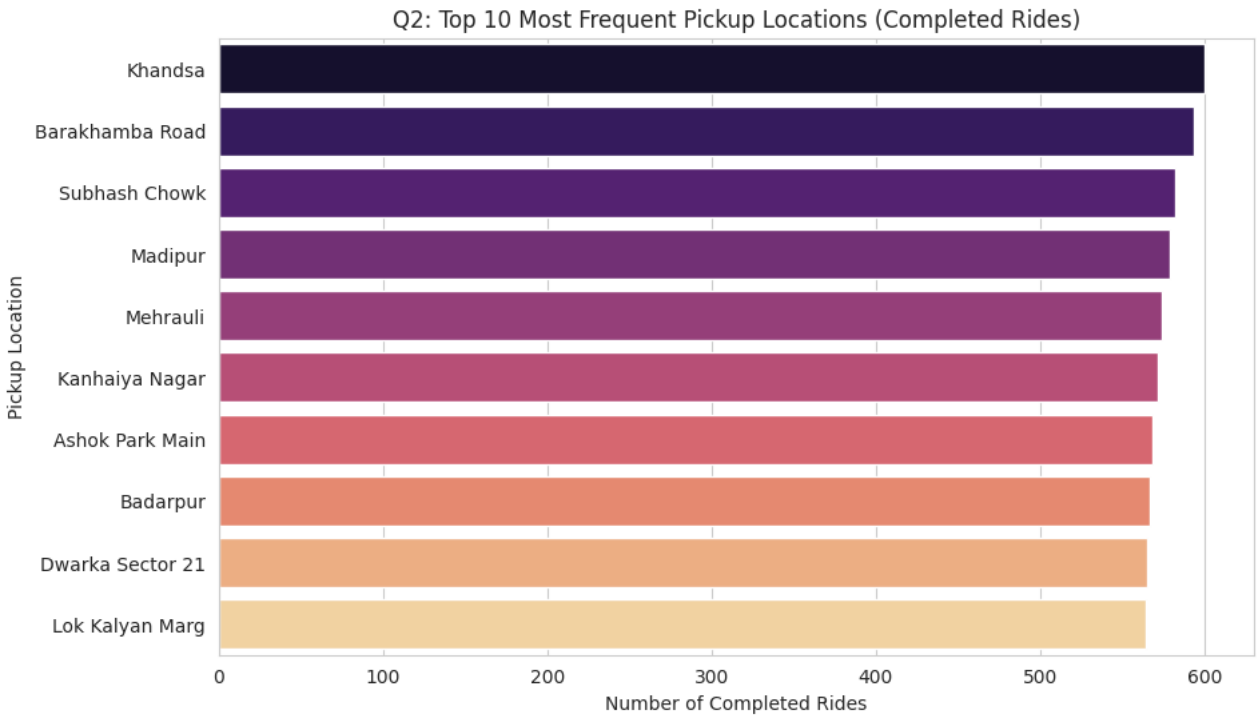
Technique: Vertical Bar Chart.

Interpretation: The Auto, Go Mini and Go Sedan segments clearly dominate bookings, establishing them as the core market drivers. This indicates high customer preference and where the bulk of service capacity should be allocated.

Insight: The platform's demand is heavily concentrated in the Auto, Go Mini and GoSedan categories. Operational and marketing efforts should focus on maintaining service quality and driver availability within these top two segments. Also, the eBike segment may require targeted incentives to grows its market share or a review of its current service viability.

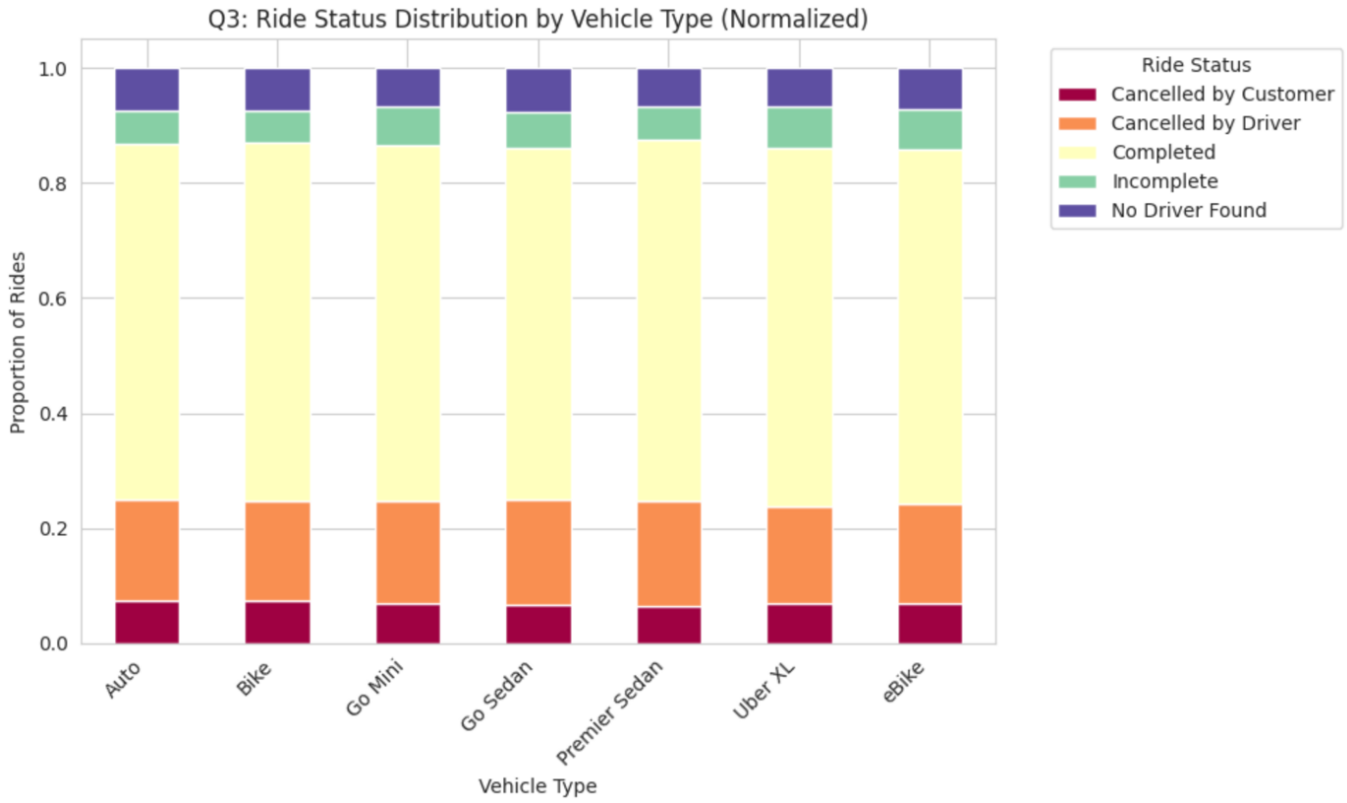


Question 2: Most frequent pick-up locations for completed rides
Relevance: Vital for operational efficiency and strategic resource allocation to minimize customer wait times.
Technique: Horizontal Bar Chart (Top 10 Locations).
Interpretation: Karol Bagh, Lajpat Nagar, and Palam Vihar lead in completed ride volume. Demand is heavily localized in these few urban centers.
Insight: the demand for ride services is heavily localized in a few high-traffic urban centres. The platform should ensure high driver density and optimal vechicle mix (sedans and autos) in the top 3 locations.



Question 3: How does the ride status distribution vary across the different vehicle types?
Relevance: Assesses service quality and identifies operational bottlenecks linked to vehicle inventory.
Technique: Normalized Stacked Bar Chart (proportional comparison).
Interpretation: The most significant finding is the remarkable consistency in the distribution of ride statuses across all vehicle types. The proportions of completed, cancelled, or unfulfilled rides are almost identical from "Auto" to "eBike."
Insight: The “completed drives” is the largest segment in every bar, indicating that most rides are completed successfully. This proportion is stable at approximately 60% to 65% across all vehicle types. The “no driver found” status represents a notable and consistent share, around 10% to 15%, for every vehicle type. This suggests a general, systemic issue with driver availability or high unmet demand, rather than a problem specific to a particular vehicle type.

The graph strongly suggests that vehicle type is not a determinant factor in the success or final status of a ride. Operational performance, user experience, and driver availability metrics appear to be uniform across the entire fleet.



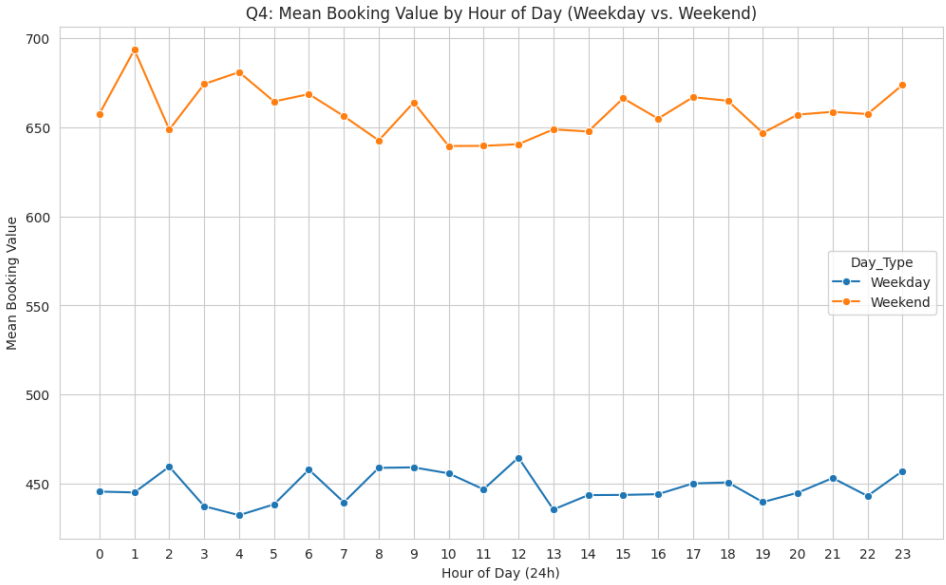
Question 4: What is the hourly trend of booking value and how does it compare across weekday vs. weekends?

Relevance: Crucial for revenue management, dynamic pricing, and optimizing driver supply allocation.

Technique: Line Plot with Grouping (Hue).

Interpretation: The Weekday pattern exhibits pronounced peaks at 06:00AM and 12:00PM, characteristic of commuting. The Weekend pattern is flatter, with later peaks (1-2 AM) and around 09:00AM, 03:00PM and 05:00PM for leisure travel.

Insight: the highest value transactions occur during weekday rush hours (08:00 AM and 12:00 PM), indicating high, time-sensitive demand for longer work commutes. The platform should apply surcharge pricing and driver bonuses during these four weekday peak hours to maximize revenue capture. On weekends, the focus should shift to managing driver supply during early-morning and afternoon social/leisure travel.



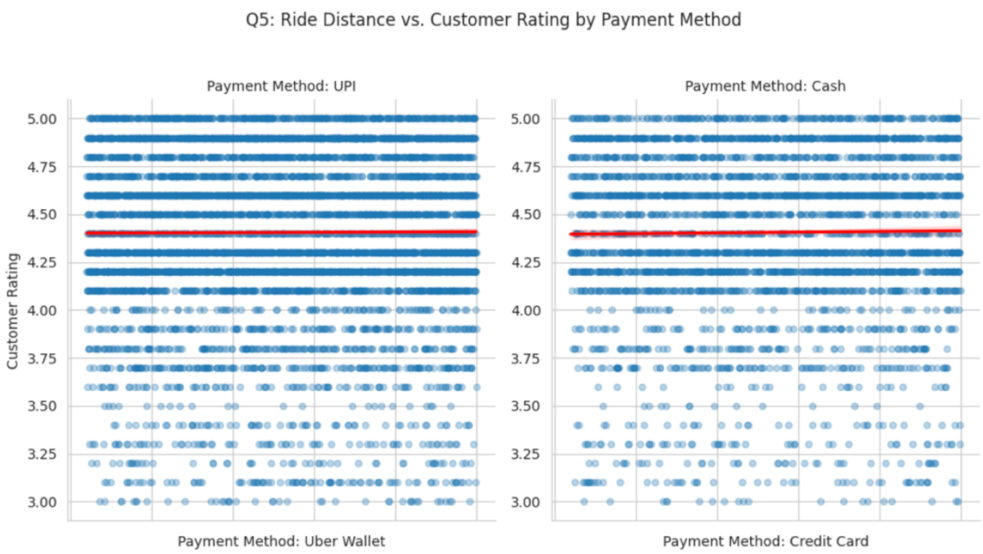
Question 5: Is there a correlation between "Ride distance" and "Customer rating", and how does it vary by "Payment method"?

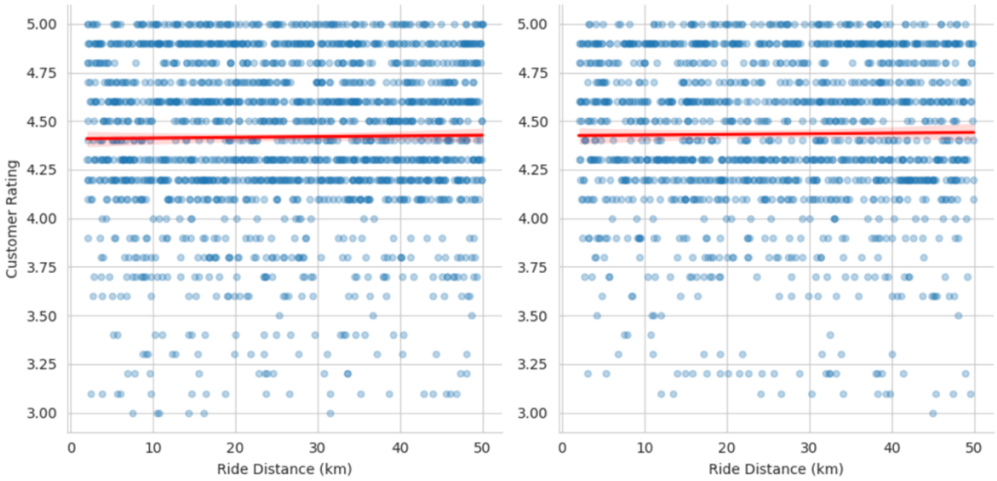
Relevance: Mandatory correlation requirement. Determines if trip length influences customer satisfaction.

Technique: Facet Grid of Scatter Plots with Regression Lines.

Interpretation: Across all payment methods, the regression line is nearly flat, indicating a negligible or extremely weak linear correlation between ride distance and customer rating. This confirms that longer trips do not inherently lead to better or worse satisfaction.

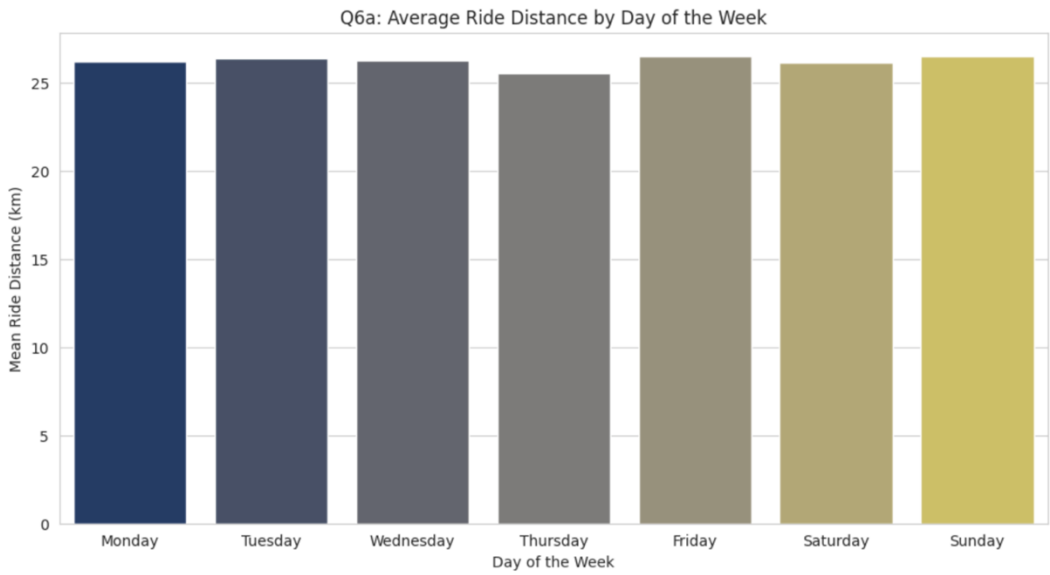
Insight: service quality, as perceived by customers, is decoupled from trip length. There are other factors than distance, such as waiting time, driver behaviour or vehicle cleanliness, more likely to be the true drivers of satisfaction. The platform could prioritize improving these non-distance-related quality metrics across all payment methods to boost customer rating universally.

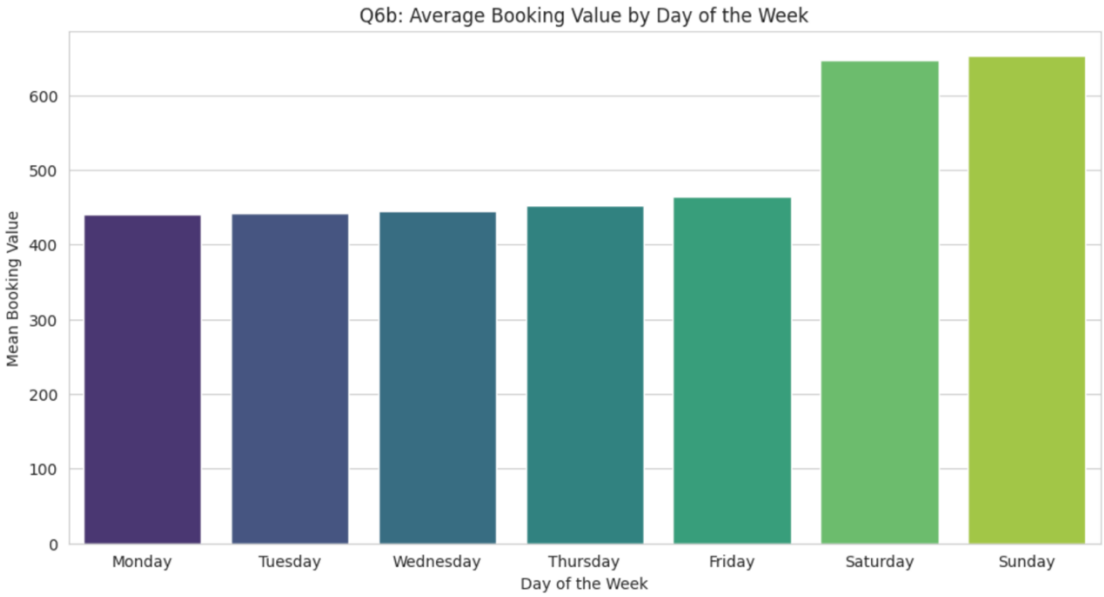




Question 6: Which day of the week has the highest average ride distance, and how does this impact average booking value?

Relevance: Strategic scheduling and profitability analysis linking trip length to revenue.
Technique: Two Separate Vertical Bar Charts (Mean Distance vs. Mean Value).
Interpretation: Both charts show that Saturday and Sunday have the highest mean ride distances (leisure travel) and, consequently, the highest average booking value per ride. Mid-weekdays have the lowest values.
Insight: weekends, specially Saturday and Sunday, represent the highest-value trips for the platform due to the significantly longer average distances travelled. To capitalize on this, the platform could introduce specific weekend driver incentives or loyalty programs based on high-distance trips. This could ensure that sufficient driver supply is available on weekends.





Conclusion

The execution of this project, combining rigorous data pre-processing with comprehensive visualization, successfully transformed a raw operational dataset into a source of actionable strategic intelligence.

Summary of Key Findings

The analysis identified two primary strategic directives. Firstly, Operational Focus must target the high service failure rates in the “No-Driver found” (Q3) and maintain supply in high-demand zones (Q2). Secondly, Revenue Optimization should aggressively leverage dynamic pricing during the high-value weekday commuting peaks (8 AM/12PM) and weekend peaks (01:00AM and afternoon), as well as weekend long-distance travel (Q4, Q6). Critically, the correlation analysis (Q5) showed that satisfaction is decoupled from distance, suggesting the platform must prioritize non-distance quality metrics like wait times and driver conduct to improve customer ratings.

Reflection on Challenges

The main technical challenge was the structural complexity of missing data, necessitating the two-stage process of filtering to 'Completed' rides, followed by rigorous IQR-based outlier removal. This ensured statistical reliability. The project demonstrated proficiency in Python's data manipulation and complex visualization techniques (Facet Grids, Normalized Stacked Bar Charts) for generating decisive business intelligence and providing clear, data-driven directives for improving the platform’s operations.

```
Final Cleaned (Completed Rides) DataFrame Information:
<class 'pandas.core.frame.DataFrame'>
Index: 93000 entries, 2 to 149999
Data columns (total 25 columns):
 #   Column                              Non-Null Count  Dtype
---  ------                              --------------  -----
 0   Date                                93000 non-null  datetime64[ns]
 1   Time                                93000 non-null  object
 2   Booking_ID                          93000 non-null  object
 3   Ride_Status                         93000 non-null  object
 4   Customer_ID                         93000 non-null  object
 5   Vehicle_Type                        93000 non-null  object
 6   Pickup_Location                     93000 non-null  object
 7   Drop_Location                       93000 non-null  object
 8   Avg_VTAT                            93000 non-null  float64
 9   Avg_CTAT                            93000 non-null  float64
 10  Cancelled_by_Customer               93000 non-null  float64
 11  Reason_for_cancelling_by_Customer   93000 non-null  object
 12  Cancelled_by_Driver                 93000 non-null  float64
 13  Driver_Cancellation_Reason          93000 non-null  object
 14  Incomplete_Rides                    93000 non-null  float64
 15  Incomplete_Rides_Reason             93000 non-null  object
 16  Booking_Value                       93000 non-null  float64
 17  Ride_Distance                       93000 non-null  float64
 18  Driver_Ratings                      93000 non-null  float64
 19  Customer_Rating                     93000 non-null  float64
 20  Payment_Method                      93000 non-null  object
 21  Timestamp                           93000 non-null  datetime64[ns]
 22  Day_of_Week                         93000 non-null  object
 23  Hour_of_Day                         93000 non-null  int64
 24  Day_Type                            93000 non-null  object
dtypes: datetime64[ns](2), float64(9), int64(1), object(13)
memory usage: 18.4+ MB

Descriptive Statistics for Key Metrics:
       Booking_Value  Ride_Distance  Driver_Ratings  Customer_Rating
count   93000.000000   93000.000000    93000.000000     93000.000000
mean      508.178215      26.000493        4.230992         4.404584
std       396.058769      13.824176        0.436871         0.437819
min        50.000000       2.000000        3.000000         3.000000
25%       234.000000      14.100000        4.100000         4.200000
50%       414.000000      26.020000        4.300000         4.500000
75%       689.000000      37.942500        4.600000         4.800000
max      4277.000000      50.000000        5.000000         5.000000
```